

**LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 14**



**DISUSUN OLEH:
RAFI IMAM NASRULLAH
109082530010
S1 IF-12-02**

**DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng**

**PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026**

SOAL LATIHAN

1. SOALA

Source Code:

```
package main

import "fmt"

const NMAX = 1000000

type arrInt [NMAX]int

func selectionSort(T *arrInt, n int) {
    var t, i, j, idxMin int
    i = 1
    for i <= n-1 {
        idxMin = i - 1
        j = i
        for j < n {
            if T[idxMin] > T[j] {
                idxMin = j
            }
            j = j + 1
        }
        t = T[idxMin]
        T[idxMin] = T[i-1]
        T[i-1] = t
        i = i + 1
    }
}

func main() {
    var n, m int
    fmt.Scan(&n)

    var daerah int
    daerah = 0
    for daerah < n {
        fmt.Scan(&m)
        var rumah arrInt
        var k int
        k = 0
        for k < m {
            fmt.Scan(&rumah[k])
            k = k + 1
        }

        selectionSort(&rumah, m)
```

```

    k = 0
    for k < m {
        if k > 0 {
            fmt.Print(" ")
        }
        fmt.Print(rumah[k])
        k = k + 1
    }
    fmt.Println()
    daerah = daerah + 1
}
}

```

Output:

The screenshot shows a Go IDE with a file explorer on the left containing folders like 'data.go', 'Soal1.go', 'LAPRAK', 'coba', and 'TUBES'. The main editor displays the code for 'Soal1.go', which includes a constant 'NPAX = 1000000', a function 'selectionSort' implementing the Selection Sort algorithm, and a 'main' function. The terminal at the bottom shows the command 'go run "c:\Users\rafi imam\OneDrive\文档\LAPRAK\Modul 14\Soal1.go"' and its output: '5 2 1 7 9 13', '1 2 7 9 13', '6 189 15 27 39 75 133', '15 27 39 75 133 189', '3 4 9 1', '1 4 9'. A small window titled 'NAMA' is also visible, displaying 'NAMA : Rafi Imam Nasrullah' and 'NIM : 109082530010'.

Deskripsi :

Program tersebut digunakan untuk membaca data nomor rumah pada beberapa daerah, kemudian mengurutkan nomor rumah tersebut dari kecil ke besar menggunakan metode Selection Sort. Program dimulai dengan membaca jumlah daerah, lalu untuk setiap daerah program membaca jumlah rumah dan nomor rumahnya. Semua nomor rumah disimpan ke dalam array, kemudian fungsi selectionSort dipanggil untuk mengurutkan data secara ascending. Metode Selection Sort bekerja dengan cara mencari nilai paling kecil pada array lalu menukarnya ke posisi depan, dan proses ini dilakukan berulang sampai seluruh data terurut. Setelah selesai diurutkan, program menampilkan nomor rumah yang sudah tersusun rapi dari nilai terkecil ke terbesar.

2. SOALA

Source Code:

```
package main

import "fmt"

func main() {
    var n int
    fmt.Scan(&n)

    for i := 0; i < n; i++ {

        var jumlah int
        fmt.Scan(&jumlah)

        var angka int
        var ganjil [100]int
        var genap [100]int
        var g, e int

        for j := 0; j < jumlah; j++ {
            fmt.Scan(&angka)

            if angka%2 == 1 {
                ganjil[g] = angka
                g++
            } else {
                genap[e] = angka
                e++
            }
        }

        for a := 0; a < g-1; a++ {
            for b := a + 1; b < g; b++ {
                if ganjil[a] > ganjil[b] {
                    ganjil[a], ganjil[b] = ganjil[b], ganjil[a]
                }
            }
        }

        for a := 0; a < e-1; a++ {
            for b := a + 1; b < e; b++ {
                if genap[a] < genap[b] {
                    genap[a], genap[b] = genap[b], genap[a]
                }
            }
        }

        for j := 0; j < g; j++ {
```

```

        fmt.Print(ganjil[j], " ")
    }

    for j := 0; j < e; j++ {
        fmt.Print(genap[j], " ")
    }

    fmt.Println()
}
}

```

Output:

```

func main() {
    var jumlah int
    fmt.Scan(&jumlah)

    var angka int
    var ganjil [100]int
    var genap [100]int
    var g, e int

    for j := 0; j < jumlah; j++ {
        fmt.Scan(&angka)

        if angka%2 == 1 {
            ganjil[g] = angka
            g++
        } else {
            genap[e] = angka
            e++
        }
    }

    for a := 0; a < g-1; a++ {
        for b := a+1; b < g; b++ {
            if ganjil[a] > ganjil[b] {
                ganjil[a], ganjil[b] = ganjil[b], ganjil[a]
            }
        }
    }

    for a := 0; a < e-1; a++ {
        for b := a+1; b < e; b++ {
            if genap[a] < genap[b] {
                genap[a], genap[b] = genap[b], genap[a]
            }
        }
    }

    for i := 0; i < g; i++ {
        fmt.Print(ganjil[i], " ")
    }

    for i := 0; i < e; i++ {
        fmt.Print(genap[i], " ")
    }

    fmt.Println()
}

```

Terminal Output:

```

PS C:\Users\rafi.imam\OneDrive\Documents\LAPRAK> go run "c:\Users\rafi.imam\OneDrive\Documents\LAPRAK\Modul 14\Soal2.go"
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4

```

Deskripsi

Program tersebut digunakan untuk mengelompokkan nomor rumah menjadi bilangan ganjil dan genap, kemudian mengurutkannya dengan aturan yang berbeda. Program pertama-tama membaca jumlah daerah, lalu untuk setiap daerah membaca jumlah nomor rumah beserta nilainya. Bilangan ganjil disimpan ke dalam array ganjil, sedangkan bilangan genap disimpan ke dalam array genap. Setelah itu, array ganjil diurutkan secara menaik dari kecil ke besar, sedangkan array genap diurutkan secara menurun dari besar ke kecil menggunakan proses pertukaran data di dalam perulangan. Setelah seluruh data selesai diurutkan, program mencetak semua bilangan ganjil terlebih dahulu kemudian dilanjutkan dengan bilangan genap sesuai aturan yang diminta soal.

3. SOALA

Source Code:

```
package main

import "fmt"

const NMAX = 1000000

type arrInt [NMAX]int

func insertionSortAsc(T *arrInt, n int) {
    var temp, i, j int
    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]
        for j > 0 && temp < T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }
        T[j] = temp
        i = i + 1
    }
}

func main() {
    var data arrInt
    var n int
    n = 0

    var angka int
    fmt.Scan(&angka)

    for angka != -5313 {
        if angka == 0 {
            insertionSortAsc(&data, n)

            var median int
            if n%2 != 0 {
                median = data[n/2]
            } else {
                median = (data[n/2-1] + data[n/2]) / 2
            }
            fmt.Println(median)
        } else {
            data[n] = angka
        }
    }
}
```

```

        n = n + 1
    }
    fmt.Scan(&angka)
}
}

```

Output:

The screenshot shows a Go IDE with the following components:

- EXPLORER:** Shows a project structure with folders like 'LAPRAK' and 'TUBES', and several Go files (Soal1.go to Soal5.go).
- MODIFIED EDITORS:** Shows the source code for 'Modul 14' (Soal3.go), which implements an insertion sort algorithm.
- TERMINAL:** Shows the command 'go run "c:\Users\rafi_imam\OneDrive\文档\LAPRAK\Modul 14\tempCodeRunnerF...' and the output '7 23 11 0 5 19 2 29 3 13 17 0 -5313'.
- OUTPUT:** A small window titled 'NAMA' displays the user's name 'Rafi Imam Nasrullah' and NIM '109082530010'.

Deskripsi:

Program tersebut digunakan untuk mencari nilai median dari sekumpulan angka yang dimasukkan pengguna. Program akan terus menerima input angka sampai pengguna memasukkan nilai -5313 sebagai tanda berhenti. Setiap angka selain 0 akan disimpan ke dalam array. Jika pengguna memasukkan angka 0, maka semua data yang sudah tersimpan akan diurutkan terlebih dahulu menggunakan metode Insertion Sort secara menaik. Setelah data terurut, program menghitung median, yaitu nilai tengah dari data. Jika jumlah data ganjil maka median diambil langsung dari elemen tengah, sedangkan jika jumlah data genap maka median diperoleh dari rata-rata dua nilai tengah. Setelah median dihitung, hasilnya langsung ditampilkan ke layar.

4. SOAL B

Source Code:

```
package main

import "fmt"

const NMAX = 1000000

type arrInt [NMAX]int

func insertionSortAsc(T *arrInt, n int) {
    var temp, i, j int
    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]
        for j > 0 && temp < T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }
        T[j] = temp
        i = i + 1
    }
}

func main() {
    var data arrInt
    var n int
    n = 0

    var angka int
    fmt.Scan(&angka)

    for angka >= 0 {
        data[n] = angka
        n = n + 1
        fmt.Scan(&angka)
    }

    insertionSortAsc(&data, n)

    var k int
    k = 0
    for k < n {
        if k > 0 {
            fmt.Print(" ")
        }
    }
```



```

        fmt.Print(data[k])
        k = k + 1
    }
    fmt.Println()

    if n <= 1 {
        fmt.Println("Data berjarak tidak tetap")
    } else {
        jarak := data[1] - data[0]
        tetap := true
        k = 1
        for k < n-1 {
            if data[k+1]-data[k] != jarak {
                tetap = false
            }
            k = k + 1
        }

        if tetap {
            fmt.Printf("Data berjarak %d\n", jarak)
        } else {
            fmt.Println("Data berjarak tidak tetap")
        }
    }
}

```

Output:

```

package main

import "fmt"

const MAX = 1000000

type arrInt [MAX]int

func insertionSortAsc(r *arrInt, n int) {
    var temp, i, j int
    i = 1
    for i <= n-1 {
        j = i
        temp = r[j]
        for j > 0 && temp < r[j-1] {
            r[j] = r[j-1]
            j = j - 1
        }
        r[j] = temp
        i = i + 1
    }
}

func main() {
    var data arrInt
    // ... = data
}

```

```

PS C:\Users\rafi\OneDrive\TUGAS\TUBES\TUBES> go run "C:\Users\rafi\OneDrive\TUGAS\TUBES\TUBES\Modul 14\Soal4.go"
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 20 32 40
Data berjarak tidak tetap
PS C:\Users\rafi\OneDrive\TUGAS\TUBES\TUBES>

```

Deskripsi :

Program tersebut digunakan untuk membaca sekumpulan bilangan, kemudian mengurutkannya dari kecil ke besar menggunakan metode Insertion Sort. Program menerima input angka selama nilainya masih lebih besar atau sama dengan nol, sedangkan angka negatif digunakan sebagai tanda berhenti memasukkan data. Setelah

semua data selesai dimasukkan, program mengurutkan seluruh angka secara ascending lalu menampilkannya ke layar. Selanjutnya program memeriksa apakah selisih antar angka yang berurutan memiliki nilai yang sama atau tidak. Jika semua selisih sama, maka program menampilkan bahwa data memiliki jarak tetap beserta besar jaraknya. Namun jika ada selisih yang berbeda, maka program akan menampilkan bahwa data berjarak tidak tetap.

5. SOAL B

Source Code:

```
package main

import (
    "fmt"
)

const nMax = 7919

type Buku struct {
    id      string
    judul   string
    penulis string
    penerbit string
    eksemplar int
    tahun   int
    rating  int
}

type DaftarBuku [nMax]Buku

var pustaka DaftarBuku
var nPustaka int

func DaftarkanBuku(n int) {
    var i int
    i = 0
    for i < n {
        fmt.Scan(&pustaka[i].id)
        fmt.Scan(&pustaka[i].judul)
        fmt.Scan(&pustaka[i].penulis)
        fmt.Scan(&pustaka[i].penerbit)
        fmt.Scan(&pustaka[i].eksemplar)
        fmt.Scan(&pustaka[i].tahun)
        fmt.Scan(&pustaka[i].rating)
        i = i + 1
    }
}
```

```

    nPustaka = n
}

func CetakTerfavorit(n int) {
    var idxMax, i int
    idxMax = 0
    i = 1
    for i < n {
        if pustaka[i].rating > pustaka[idxMax].rating {
            idxMax = i
        }
        i = i + 1
    }
    fmt.Println("Buku Terfavorit:")
    fmt.Println("Judul   :", pustaka[idxMax].judul)
    fmt.Println("Penulis  :", pustaka[idxMax].penulis)
    fmt.Println("Penerbit :", pustaka[idxMax].penerbit)
    fmt.Println("Tahun    :", pustaka[idxMax].tahun)
}

func UrutBuku(n int) {
    var temp Buku
    var i, j int
    i = 1
    for i <= n-1 {
        j = i
        temp = pustaka[j]
        for j > 0 && temp.rating > pustaka[j-1].rating {
            pustaka[j] = pustaka[j-1]
            j = j - 1
        }
        pustaka[j] = temp
        i = i + 1
    }
}

func Cetak5Terbaru(n int) {
    fmt.Println("\n5 Buku dengan Rating Tertinggi:")
    limit := 5
    if n < 5 {
        limit = n
    }
    var i int
    i = 0
    for i < limit {
        fmt.Printf("%d. %s (Rating: %d)\n", i+1, pustaka[i].judul, pustaka[i].rating)
        i = i + 1
    }
}

```

```

func CariBuku(n int, r int) {
    var lo, hi, mid int
    lo = 0
    hi = n - 1
    var found bool
    found = false
    var idxFound int
    idxFound = -1

    for lo <= hi {
        mid = (lo + hi) / 2
        if pustaka[mid].rating == r {
            found = true
            idxFound = mid
            lo = hi + 1
        } else if pustaka[mid].rating < r {
            hi = mid - 1
        } else {
            lo = mid + 1
        }
    }

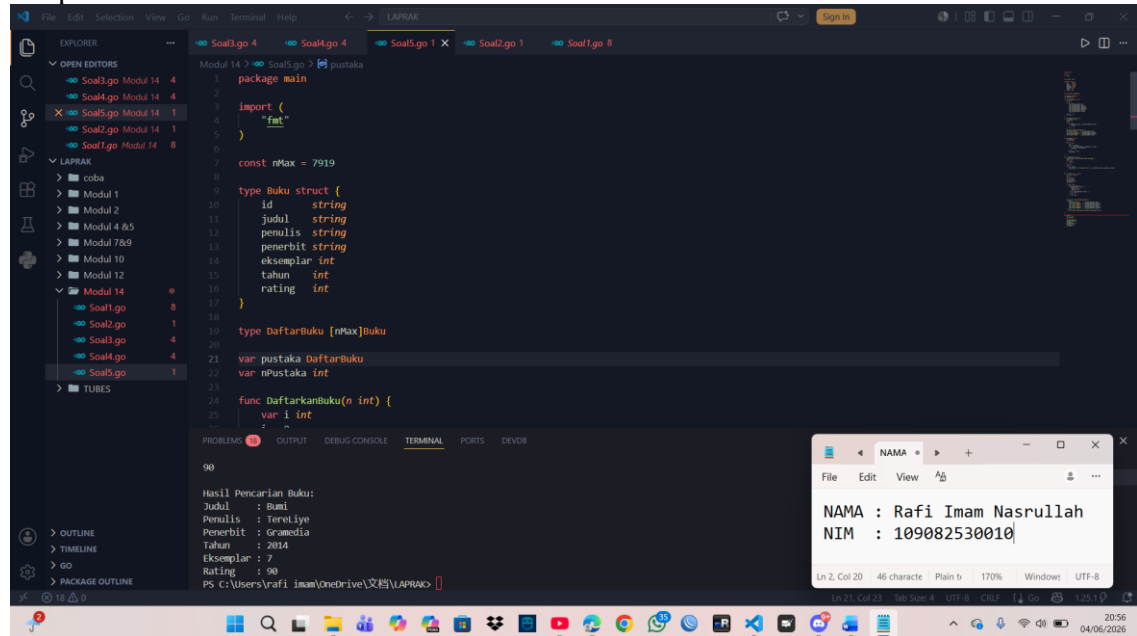
    fmt.Println("\nHasil Pencarian Buku:")
    if found {
        fmt.Println("Judul   :", pustaka[idxFound].judul)
        fmt.Println("Penulis  :", pustaka[idxFound].penulis)
        fmt.Println("Penerbit :", pustaka[idxFound].penerbit)
        fmt.Println("Tahun    :", pustaka[idxFound].tahun)
        fmt.Println("Eksemplar :", pustaka[idxFound].eksemplar)
        fmt.Println("Rating   :", pustaka[idxFound].rating)
    } else {
        fmt.Println("Tidak ada buku dengan rating seperti itu")
    }
}

func main() {
    var n, r int
    fmt.Scan(&n)

    DaftarkanBuku(n)
    CetakTerfavorit(n)
    UrutBuku(n)
    Cetak5Terbaru(n)
    fmt.Scan(&r)
    CariBuku(n, r)
}

```

Output:



```
package main

import (
    "fmt"
)

const nMax = 7919

type Buku struct {
    id      string
    judul   string
    penulis string
    penerbit string
    eksemplar int
    tahun   int
    rating   int
}

type DaftarBuku [nMax]Buku

var pustaka DaftarBuku
var nPustaka int

func DaftarkanBuku(n int) {
    var i int
    for i = 0; i < n; i++ {
        // ... (code for adding books) ...
    }
}
```

90

Hasil Pencarian Buku:
Judul : Bumi
Penulis : Terejiye
Penerbit : Gramedia
Tahun : 2014
Eksemplar : 7
Rating : 90

Deskripsi :

Program tersebut digunakan untuk mengelola data buku pada sebuah perpustakaan. Program dimulai dengan membaca jumlah buku, kemudian setiap data buku dimasukkan sesuai atribut seperti id, judul, penulis, penerbit, jumlah eksemplar, tahun terbit, dan rating. Semua data buku disimpan ke dalam array pustaka. Setelah itu, program mencari buku dengan rating tertinggi dan menampilkan informasi judul, penulis, penerbit, serta tahunnya sebagai buku terfavorit. Selanjutnya data buku diurutkan secara menurun berdasarkan rating menggunakan metode Insertion Sort. Setelah terurut, program menampilkan lima buku dengan rating tertinggi atau kurang dari lima jika jumlah buku tidak mencukupi. Pada bagian akhir, pengguna memasukkan sebuah rating untuk dicari. Program kemudian melakukan pencarian menggunakan metode binary search dan menampilkan data buku yang memiliki rating tersebut. Jika tidak ditemukan, program akan menampilkan pesan bahwa tidak ada buku dengan rating seperti itu.